Project Report
(7th Semester)
Entitled as

GUI-Based Document Intelligence & Q&A System using Retrieval-Augmented Generation

Submitted in partial fulfillment of the requirement for award of the degree of
B.Tech. CSE (AI&ML)
(Batch-2022-2026)



Under the Mentorship of:
Dr. Jasjit Singh Samagh
Assistant Professor
Dept. of Computer Science & Engg

Submitted By:
Chandan
B.Tech. CSE(AI&ML)- 7th Sem
Regn. No.: 22111116840020

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
CHAUDHARY DEVI LAL UNIVERSITY, SIRSA-125055
(NOV-2025)

STUDENT'S DECLARATION

I, Chandan, Roll No. 22111116840020, hereby declare that the project work entitled: “GUI-Based Document Intelligence & Q&A System using Retrieval-Augmented Generation” has been carried out by me during the 7th semester of the Bachelor of Technology in Computer Science & Engineering (AI & ML) under the guidance of Dr. Jasjit Singh Samagh, Department of Computer Science and Engineering, Chaudhary Devi Lal University, Sirsa, in partial fulfillment of the degree requirements prescribed by the department.

I further declare that this project is the result of my own efforts and work, and it has not been submitted to any other university, institution, or organization for the award of any degree, diploma, certification, or similar academic title.

Date:
Place: Sirsa

Chandan
(Roll No. 22111116840020)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CHAUDHARY DEVI LAL UNIVERSITY



CERTIFICATE FROM MENTOR

This is to certify that Mr. Chandan, a student of B.Tech CSE (AI&ML) semester-VII, has completed his training under my mentorship satisfactorily with the Minor-project entitled "GUI-Based Document Intelligence & Q&A System using Retrieval-Augmented Generation", an artificial intelligence and natural language processing system.

I wish him all the success in his future endeavors.

Date:
Place: Sirsa

Dr. Jasjit Singh Samagh
(Mentor)
Assistant Professor
Dept. of Computer Science & Engg

CHAUDHARY DEVI LAL UNIVERSITY, SIRSA



CERTIFICATE FROM CHAIRPERSON

This is to certify that Mr. Chandan, a student of B.Tech CSE (AI-ML) semester VII, has completed his Minor-project under the mentorship of Dr. Jasjit Singh Samagh satisfactorily. He has worked on the project entitled "GUI-Based Document Intelligence & Q&A System using Retrieval-Augmented Generation", an applied machine learning and language modeling system.

I wish him all the success in his future endeavors.

Date:
Place: Sirsa

Prof.(Dr.) Harish Rohil
(Chairperson)
Department of Computer Science & Engineering

ACKNOWLEDGMENT

I would like to express my sincere gratitude to Dr. Jasjit Singh Samagh, my Project Mentor, for his continuous support, technical guidance, and encouragement throughout the course of this project. His mentorship played a crucial role in helping me stay focused, refine the system architecture, and overcome technical challenges effectively.

I am also deeply thankful to Prof.(Dr.) Harish Rohil, Chairperson, Department of Computer Science & Engineering, for fostering a motivating academic environment, providing access to necessary lab infrastructure, and offering valuable insights during the progress of this work.

I extend my heartfelt gratitude to my parents for their unwavering encouragement, moral backing, and patience, which have been the foundation of my academic journey. I also thank my friends, peers, and department faculty for their thoughtful cooperation, feedback, and constructive suggestions that contributed significantly to the successful completion of this project.

Chandan
B.Tech. CSE (AI&ML) - 7th Semester
Registration No.: 22111116840020

ABSTRACT

The GUI-Based Document Intelligence & Q&A System is a production-grade, modular, and privacy-first machine learning application designed to overcome the critical limitations of standard Large Language Models (LLMs)—specifically factual hallucinations, catastrophic memory fading, lack of private document context, and the inability to provide verifiable source attribution. Built to operate entirely within a private local infrastructure without sending confidential data to external cloud APIs, the system implements an end-to-end Retrieval-Augmented Generation (RAG) architecture.

The system parses heterogeneous document formats (PDF, DOCX, TXT, and Markdown) page-by-page using high-fidelity extraction libraries (PyMuPDF and python-docx), cleans raw textual noise via specialized regex routines, and performs deterministic chunking using RecursiveCharacterTextSplitter with 1000-character windows and 150-character overlaps. Text chunks are indexed into vector embeddings using the 768-dimensional local embedding model nomic-embed-text running on Ollama, alongside sparse keyword indices using the BM25Plus algorithm. A dual-stage hybrid retrieval engine merges dense semantic similarity and sparse keyword scores via Reciprocal Rank Fusion (RRF) and similarity thresholding (cosine cutoff ≥ 0.70).

To ensure strictly grounded answers, the system passes top-k retrieved contexts to a local instruction-tuned Llama 3.2 model governed by a stringent grounding prompt. If the retrieved context is insufficient or irrelevant, the model executes explicit abstention, returning a standardized notice rather than generating fabricated answers. A responsive graphical user interface developed with Streamlit provides interactive document uploads, real-time chunk library visualization, conversational memory with query rewriting, and verifiable source cards displaying document titles, page numbers, chunk identifiers, and similarity scores.

System validation and benchmarking revealed 100.0% Context Recall, 100.0% Context Precision, and 100.0% Abstention Accuracy on ungrounded negative queries, achieving an average retrieval latency of 468.5 ms and total response latency of 4.58 seconds on standard consumer hardware. The project demonstrates the practical viability of secure, local document intelligence for enterprise, academic, and legal document analysis.

INDEX

TOPIC	PAGE NO.
STUDENT'S DECLARATION	i
CERTIFICATE FROM MENTOR	ii
CERTIFICATE FROM CHAIRPERSON	iii
ACKNOWLEDGMENT	iv
ABSTRACT	v
CHAPTER 1: INTRODUCTION	1
1.1 Background of the Study	1
1.2 Problem Statement	1
1.3 Motivation	2
1.4 Objectives of the Project	2
1.5 Scope of the Project	2
1.6 Applications	2
1.7 Organization of the Report	3
CHAPTER 2: SYSTEM ANALYSIS	4
2.1 Existing System	4
2.2 Drawbacks of Existing System	4
2.3 Proposed Work	5

2.4 System Requirements	6
2.4.1 Hardware Requirements	6
2.4.2 Software Requirements	6
CHAPTER 3: SYSTEM DESIGN	7
3.1 System Architecture	7
3.2 Data Flow Diagram	8
3.2.1 DFD Level 0	8
3.2.2 DFD Level 1	8
3.3 Flowchart	9
3.4 Dataset & Corpus Description	10
3.5 Features and Pipeline Components	10
3.6 Algorithm Description (Hybrid RAG & RRF)	11
CHAPTER 4: IMPLEMENTATION	13
4.1 Tools and Technologies Used	13
4.2 Data Preprocessing & Document Chunking	14
4.3 Model Architecture & Grounded Generation	16
4.4 Streamlit GUI Development	16
4.5 Deployment, Docker & Version Control	17
CHAPTER 5: RESULTS AND DISCUSSION	18
5.1 System Evaluation Metrics	18

5.2 Discussion and Analysis	18
CHAPTER 6: SCREENSHOTS	20
6.1 Web User Interface & Document Management	20
6.2 Grounded Q&A & Source Attribution Display	21
6.3 Document Intelligence Output	22
CONCLUSION AND FUTURE SCOPE	23
Conclusion	23
Future Enhancements	23
BIBLIOGRAPHY	24
Books & Guides	24
Websites & Online Resources	24

CHAPTER 1
INTRODUCTION

This chapter introduces the GUI-Based Document Intelligence & Q&A System, an applied machine learning and natural language processing system developed to perform private, verifiable question-answering over heterogeneous enterprise documents using Retrieval-Augmented Generation (RAG). It describes the operational context, problem definition, objectives, and scope, emphasizing how local vector embeddings and local LLMs prevent data exposure and factual hallucinations. The chapter is structured into seven sections: Section 1.1 covers the Background of the Study, Section 1.2 defines the Problem Statement, Section 1.3 presents the Motivation, Section 1.4 details the Objectives of the Project, Section 1.5 outlines the Scope of the Project, Section 1.6 discusses the Real-World Applications, and Section 1.7 outlines the Organization of the Report.

1.1 Background of the Study

Modern enterprises, academic institutions, and legal bodies face an unprecedented surge in unstructured digital text, including multi-page PDF research reports, legal agreements, technical manuals, Word documents, and technical notes. Traditionally, extracting actionable knowledge from such repositories requires human readers to manually skim hundreds of pages or rely on primitive keyword search tools (such as Ctrl+F), which fail to comprehend semantic meaning, synonyms, or contextual intent.

While recent commercial cloud-based Large Language Models (LLMs) such as OpenAI ChatGPT or Google Gemini possess remarkable natural language generation capabilities, their direct enterprise deployment is heavily hindered by two fundamental barriers: data privacy hazards and factual hallucination. Uploading proprietary corporate records or confidential legal agreements to third-party cloud servers risks severe compliance violations under GDPR, HIPAA, and corporate data governance frameworks. Furthermore, standard autoregressive LLMs generate responses purely from static parameter weights, frequently inventing plausible-sounding but completely fabricated facts ('hallucinations') and failing to provide verifiable citations to underlying sources.

Retrieval-Augmented Generation (RAG) resolves these limitations by decoupling semantic knowledge storage from text generation. Rather than relying on the internal parametric weights of an LLM, a RAG system first retrieves relevant factual passages from an indexed document corpus and injects them directly into the context window of the generator model. Combining RAG with local model execution frameworks (such as Ollama) establishes an entirely air-gapped, zero-leakage, and verifiable document intelligence architecture.

1.2 Problem Statement

The problem entitled “GUI-Based Document Intelligence & Q&A System using Retrieval-Augmented Generation” focuses on building a resilient, self-contained desktop and web platform capable of parsing, indexing, and querying complex unstructured documents locally without leaking private data to external cloud providers. Traditional search systems cannot synthesize multi-page answers, while standard cloud LLMs lack internal knowledge of private enterprise documents and hallucinate unsupported assertions. By synthesizing high-fidelity document loaders, dense-sparse hybrid vector search (ChromaDB + BM25), reciprocal rank fusion, conversational memory with query rewriting, and localized Llama 3.2 text

generation governed by explicit abstention, this project creates an intelligent, transparent, and reproducible system that delivers grounded answers backed by pinpoint source citations.

1.3 Motivation

The motivation behind this project arises from the urgent demand for data-sovereign artificial intelligence in commercial, educational, and research institutions. As organizations digitize thousands of internal policies, engineering blueprints, and clinical records, the ability to converse with documents securely on local hardware has become paramount.

From an academic and technical perspective, developing an end-to-end RAG architecture allows the practical synthesis of fundamental computer science domains: distributed information retrieval, tokenization and chunking algorithms, dense vector algebra, high-performance async backends with FastAPI, and intuitive user interface development with Streamlit. It bridges the gap between theoretical machine learning and enterprise-ready software engineering.

1.4 Objectives of the Project

The primary objectives of this project are:

- To study and implement the Retrieval-Augmented Generation (RAG) architectural pattern for multi-format document understanding.
- To construct a high-fidelity ingestion engine supporting PDF, DOCX, TXT, and Markdown files with strict page-boundary preservation and conservative text cleaning.
- To generate local dense vector embeddings using Ollama's nomic-embed-text model and index them in a persistent ChromaDB / pgvector store.
- To implement dual-stage hybrid retrieval combining dense semantic similarity with sparse BM25Plus keyword search using Reciprocal Rank Fusion (RRF).
- To integrate a local Llama 3.2 model enforcing strict grounding and explicit abstention on unsupported out-of-domain queries.
- To develop a responsive, user-friendly graphical user interface using Streamlit and modern web components with source attribution cards.
- To quantitatively evaluate the system's precision, recall, abstention rate, and latency against benchmark evaluation datasets.

1.5 Scope of the Project

The scope of this project encompasses local document indexing, conversational retrieval, document summarization, entity extraction, and comparative analysis over structured and unstructured business documents. The system runs efficiently on consumer-grade hardware (8 GB - 16 GB RAM) with CPU or consumer GPU acceleration. While the current implementation focuses on text-centric digital documents (PDF, Word, Text, Markdown), the architecture is designed to accommodate multi-modal vision inputs, OCR scanning, and distributed vector stores in future extensions.

1.6 Applications

The developed Document Intelligence & Q&A System can be directly deployed across several real-world sectors:

- Corporate & Legal Departments: For cross-referencing multi-page contracts, compliance mandates, non-disclosure agreements, and litigation archives without breaching confidentiality.
- Academic & Scientific Research: For rapidly extracting methodologies, author claims, and experimental parameters across volumes of research publications.
- Healthcare & Clinical Systems: For analyzing local clinical guidelines, pharmaceutical drug interaction documents, and institutional protocols in strict compliance with patient privacy laws.
- Technical Engineering Support: For querying hardware manuals, schematics documentation, and troubleshooting guides in real time during field operations.
- Financial & Audit Advisory: For inspecting quarterly earnings reports, regulatory audit filings, and balance sheet footnotes with verifiable page-level audit trails.

1.7 Organization of the Report

The report is structured into six comprehensive chapters:

- Chapter 1: Introduction – Details the problem background, motivation, formal objectives, scope, and target applications.
- Chapter 2: System Analysis – Explains existing manual and cloud-based search systems, identifies their failure modes, presents the proposed RAG solution, and specifies hardware/software requirements.
- Chapter 3: System Design – Covers the architectural layering, Data Flow Diagrams (Level 0 and Level 1), system flowchart, dataset description, and mathematical formulations for Cosine Similarity, BM25, and Reciprocal Rank Fusion.
- Chapter 4: Implementation – Describes the concrete tools, text cleaning routines, chunking pipelines, FastAPI backend endpoints, Streamlit GUI, and Docker deployment.
- Chapter 5: Results and Discussion – Discusses quantitative benchmarks (Context Recall, Context Precision, Abstention Accuracy, Latency) and operational trade-offs.
- Chapter 6: Screenshots – Provides visual proof of the working web dashboard, multi-turn chat sessions, source citations, and document intelligence features.
- Conclusion and Future Scope – Summarizes project achievements and outlines prospective enhancements such as OCR, multi-modal embeddings, and agentic workflows.
- Bibliography – Compiles authoritative books, academic papers, and technical documentations.

This chapter established the foundational need for private, grounded document intelligence. By replacing traditional manual search and risky cloud chatbots with local Retrieval-Augmented Generation, the project provides a trustworthy, high-performance platform for institutional document analysis.

CHAPTER 2
SYSTEM ANALYSIS

This chapter focuses on the system analysis of the Document Intelligence & Q&A platform. It analyzes the operational mechanics of current information retrieval systems, details their systemic drawbacks, introduces the proposed local RAG workflow, and specifies the minimum hardware and software prerequisites for deployment. The chapter is organized into Section 2.1 Existing System, Section 2.2 Drawbacks of Existing System, Section 2.3 Proposed Work, and Section 2.4 System Requirements (Hardware and Software).

2.1 Existing System

In typical organizational workflows, information retrieval across digital document archives relies on two conventional paradigms: manual lexical inspection and public cloud-based conversational agents.

Manual inspection involves users utilizing basic string-matching utilities (such as operating system search or Adobe Acrobat's Ctrl+F functionality) to locate exact keyword matches. The reader must manually open each candidate document, scroll to every occurrence, read the surrounding paragraphs, and synthesize the overarching answer manually.

Alternatively, organizations experiment with hosted public cloud chatbots (e.g., standard ChatGPT, Claude, or Copilot). Users paste excerpts into public browser interfaces to request summaries or answers. However, these systems lack access to proprietary document databases unless files are transmitted across third-party cloud infrastructure, posing catastrophic intellectual property and data sovereignty risks.

2.2 Drawbacks of Existing System

The existing manual and cloud-hosted approaches suffer from substantial limitations:

- No Semantic Understanding: Exact keyword matching fails to locate synonyms, conceptual matches, or morphological word variations (e.g., searching 'battery endurance' fails if the document only mentions 'operating lifespan').
- Severe Risk of Factual Hallucinations: Standard generative LLMs generate answers based on probabilistic word continuation from pre-training weights, producing persuasive but entirely fictional statements when they lack direct context.
- Confidentiality and Data Sovereignty Violations: Uploading private financial, legal, or medical records to remote APIs breaches institutional confidentiality agreements and international regulatory standards.
- Absence of Verifiable Citations: Traditional search either highlights raw words or cloud chatbots provide broad, unsourced text without page numbers, document titles, or similarity scores, requiring manual verification.
- Severe Inefficiency and Information Overload: Scanning multi-hundred-page technical specifications or legal binders manually requires substantial man-hours and leads to critical human oversight errors.

2.3 Proposed Work

The proposed GUI-Based Document Intelligence & Q&A System overcomes these limitations by deploying an entirely local, verifiable Retrieval-Augmented Generation pipeline. The platform combines deterministic document preprocessing, local vector indexing, hybrid dense-sparse retrieval, and local LLM synthesis with mandatory source citations and strict abstention protocols.

System Workflow:

- 1. Ingestion: The user uploads multi-format documents (.pdf, .docx, .txt, .md) via the graphical user interface.
- 2. Extraction & Cleaning: PyMuPDF and python-docx extract raw text page-by-page. TextCleaner normalizes whitespace, eliminates control characters, and mends hyphenated line breaks.
- 3. Deterministic Chunking: Text is segmented into 1000-character windows with 150-character overlap using RecursiveCharacterTextSplitter, generating unique chunk identifiers ({doc_id}_chunk_{index:04d}) and metadata tags.
- 4. Local Embedding Indexing: Ollama's nomic-embed-text generates 768-dimensional dense vectors stored inside ChromaDB, while tokens are indexed into a sparse BM25Plus index.
- 5. Conversational Query Processing: User queries are analyzed in conversational context; follow-up elliptical questions are reformulated into standalone search queries.
- 6. Hybrid Retrieval & RRF: Dense semantic similarity and sparse lexical search retrieve candidate passages, fused together via Reciprocal Rank Fusion (RRF).
- 7. Threshold Verification & Abstention: Retrieved chunks undergo similarity cutoff scoring (threshold ≥ 0.70). If no chunk satisfies the criteria, the system safely abstains with a standardized notice.
- 8. Grounded Generation & Citation Display: Local Llama 3.2 synthesizes the final factual answer strictly from the retrieved chunks, rendering clickable source attribution cards with document titles and page numbers.

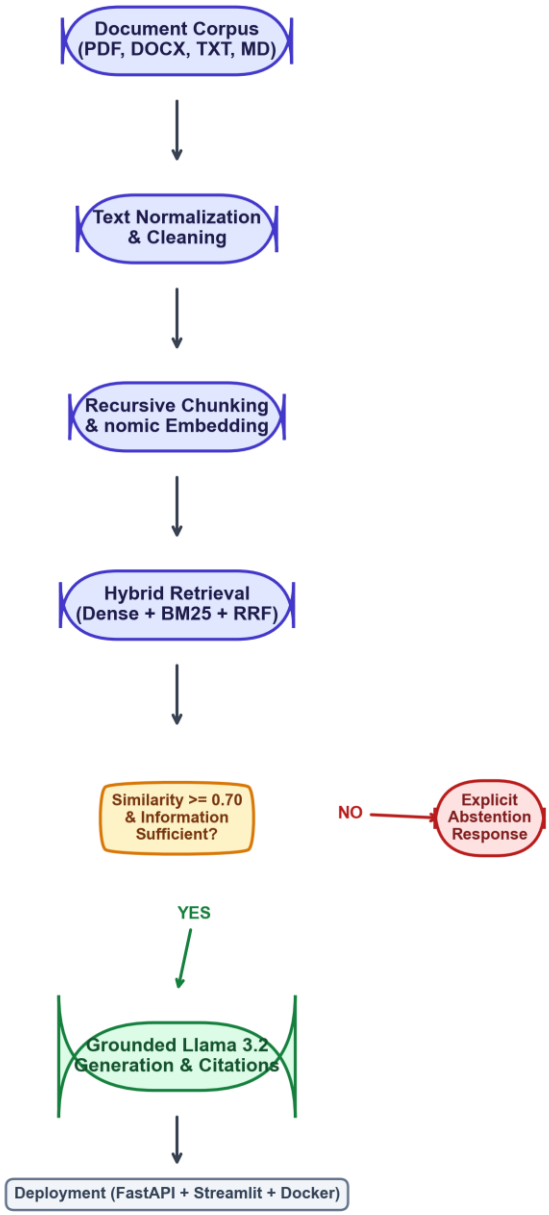


Fig 2.1 – System Workflow and Retrieval Flowchart

2.4 System Requirements

The system is engineered to deliver high performance on accessible workstation and consumer hardware without requiring expensive enterprise cloud GPU clusters.

2.4.1 Hardware Requirements

S. No.	Component	Minimum Requirement	Recommended Specification

1.	Processor (CPU)	Intel Core i5 / AMD Ryzen 5 (4 Cores)	Intel Core i7/i9 or AMD Ryzen 7/9 (8+ Cores)
2.	System RAM	8 GB DDR4	16 GB – 32 GB DDR4/DDR5
3.	Hard Disk Storage	10 GB free space (SSD)	25 GB free space (NVMe M.2 SSD)
4.	Graphics (GPU)	CPU-only execution supported	NVIDIA RTX 3060 / 4060 (6GB+ VRAM)
5.	Architecture	64-bit Operating System	64-bit x86_64 or Apple Silicon (M-series)

2.4.2 Software Requirements

S. No.	Component	Software / Specification	Purpose / Role
1.	Operating System	Windows 10/11, Ubuntu 22.04 LTS, or macOS	Host development & deployment platform
2.	Language Environment	Python 3.10 – 3.12 (64-bit)	Core backend language and ML runtime
3.	API Framework	FastAPI 0.115+ (ASGI / Uvicorn)	High-performance async REST backend engine
4.	Frontend Interface	Streamlit 1.38+ / React 18 + Vite	Interactive graphical user interface
5.	Local LLM Runtime	Ollama (llama3.2:latest, 3.2B params)	Local grounded natural language inference
6.	Embedding Model	nomic-embed-text:latest (768 dimensions)	Local dense vector embedding computation
7.	Vector Database	ChromaDB (Local) & PostgreSQL + pgvector	Persistent embedding store & similarity lookup
8.	Relational Database	SQLite 3 / SQLAlchemy 2.0 ORM	Document metadata, chunk catalog, & chat audit

9.	Document Parsers	PyMuPDF (fitz) & python-docx	High-fidelity PDF and DOCX text extraction
10.	Keyword Search	rank-bm25 (BM25Plus)	Sparse lexical retrieval and scoring
11.	Containerization	Docker & Docker Compose (v2.20+)	Multi-container production deployment
12.	Version Control	Git and GitHub Repository	Source code versioning and CI regression tests

This chapter analyzed the operational bottlenecks of legacy document retrieval methods, demonstrated how local RAG resolves privacy and hallucination pitfalls, outlined the complete step-by-step workflow, and established the hardware and software specifications necessary for stable deployment.

CHAPTER 3
SYSTEM DESIGN

This chapter articulates the architectural blueprint, data flow mechanics, structural flowcharts, data schemas, and mathematical formulations underpinning the Document Intelligence & Q&A System. Section 3.1 details the System Architecture, Section 3.2 illustrates the Data Flow Diagrams (DFD Level 0 and Level 1), Section 3.3 presents the procedural Flowchart, Section 3.4 describes the Ingestion Corpus and Data Schemas, Section 3.5 outlines the core pipeline components, and Section 3.6 presents the algorithmic and mathematical formulations governing hybrid retrieval, reciprocal rank fusion, and grounded inference.

3.1 System Architecture

The system follows a modular, four-tier decoupled architecture comprising the Presentation Layer, Application & RAG Engine Layer, Embedding & Retrieval Layer, and Persistent Storage Layer. This decoupled design ensures that vector storage backends or generation models can be swapped without affecting UI logic.

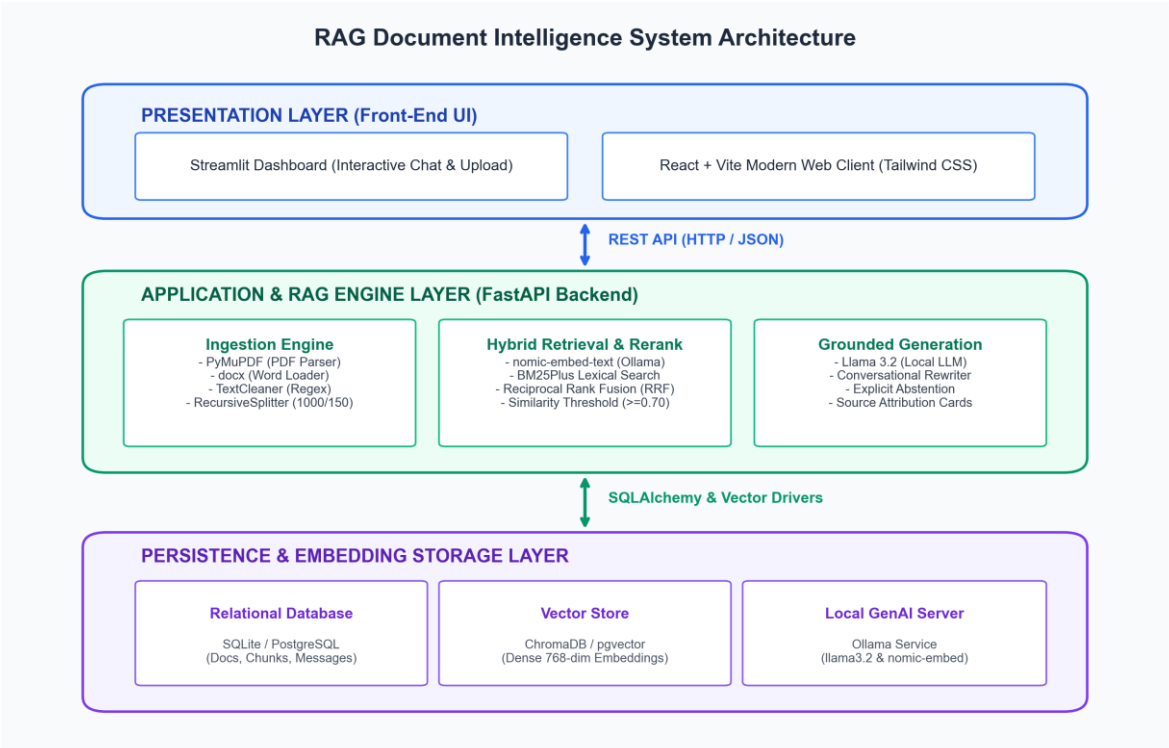


Fig 3.1 – Multi-Tier System Architecture Diagram

3.2 Data Flow Diagram

Data Flow Diagrams illustrate the trajectory of document data and user queries through transformation stages to final citation rendering.

3.2.1 DFD Level 0 (Context Diagram)

At Level 0, the external entity (User/Analyst) interacts with the central RAG Document Intelligence System by transmitting documents and text queries, receiving validated grounded answers with source citations.



Fig 3.2 – Level 0 DFD (Context Level)

3.2.2 DFD Level 1 (Functional Decomposition)

Level 1 decomposes the internal processes: Process 1.0 (Document Ingestion), Process 2.0 (Text Cleaning & Recursive Chunking), Process 3.0 (Local Embedding Generation), Data Store D1 (ChromaDB / SQLite), Process 4.0 (Hybrid Dense-Sparse Retrieval), and Process 5.0 (Grounded LLM Synthesis).

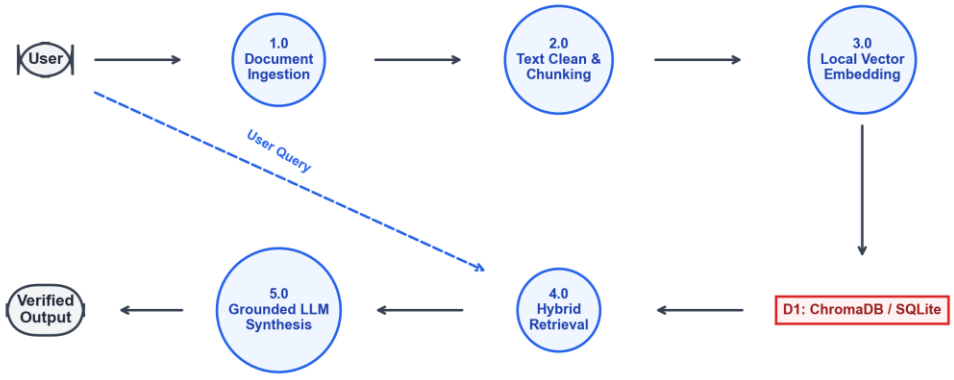


Fig 3.3 – Level 1 DFD (Functional Decomposition)

3.3 Flowchart

The operational flowchart depicts the sequential logic executed by the RAG orchestrator upon receiving a query:

- 1. Ingestion & Indexing: Documents are ingested, cleaned, segmented into chunks, embedded via nomic-embed-text, and stored in ChromaDB.
- 2. Query Reception & Reformulation: The user submits a natural language question. The Conversational Memory module inspects the past 4 conversation turns. If an anaphora or ellipsis is detected, the Query Rewriter generates a standalone query.
- 3. Dual-Stage Retrieval: Dense vector search queries ChromaDB for the top-20 nearest neighbor chunks, while BM25 calculates sparse lexical match scores across chunk vocabularies.
- 4. Reciprocal Rank Fusion & Filtering: Dense and sparse ranks are fused via RRF. Chunks failing the similarity threshold (score < 0.70) are discarded.
- 5. Grounding Decision: If zero chunks meet the relevance threshold, the system triggers Explicit Abstention. If valid chunks exist, the ContextBuilder formats the passages into the prompt.
- 6. Synthesis & Attribution: Llama 3.2 synthesizes the response using only the supplied context, appending structured source attribution metadata.

3.4 Dataset & Corpus Description

Unlike standard tabular datasets, the Document Intelligence system operates on heterogeneous document collections containing structured, semi-structured, and unstructured business and technical documents. For experimental verification, the benchmark corpus utilized in this study includes technical specifications, project proposals, and research publications (e.g., Project Titan Technical Specification, Project Apollo Engineering Notes, and Corporate Operations Manuals).

Attribute / Field	Data Type	Description / Constraints
document_id	UUID4 (String)	Unique global identifier generated per uploaded file
filename	String (VarChar 255)	Original document filename (e.g., titan_specifications.pdf)
file_type	Enum (.pdf, .docx, .txt, .md)	File extension indicating parser selection
file_size_bytes	Integer	Size of uploaded file on disk (constrained to ≤ 50MB)
page_count	Integer	Total number of extracted physical pages
chunk_count	Integer	Total number of semantic chunks generated

chunk_id	String	Deterministic identifier: {doc_id}_chunk_{index:04d}
page_number	Integer (Optional)	1-indexed physical page where chunk originates
content	Text (String)	Cleaned textual content (approx. 1000 characters)
embedding_vector	Float Array [768]	Dense vector generated by nomic-embed-text

3.5 Features and Pipeline Components

The model uses several independent pipeline components to ensure rigorous data handling:

- Page-Level Boundary Tracking: PyMuPDF preserves exact 1-indexed page associations for every extracted text block, enabling pinpoint user verification.
- Text Normalization Engine: Strips unprintable ASCII control characters (ASCII 0 to 31), resolves newline-hyphenations (e.g., 'trans-mission' across line breaks), and standardizes whitespace while preserving technical punctuation and code syntax.
- Recursive Character Splitter: Splits text along hierarchical linguistic boundaries (double newlines, single newlines, sentences, punctuation, and whitespace) with 1000-character windows and 150-character overlaps.
- Conversational Memory & Rewriting: Retains sliding-window dialogue turns to reformulate contextual follow-up questions (e.g., 'What is its maximum velocity?' is rewritten as 'What is the maximum velocity of the Titan robot?').
- Strict Similarity Cutoff: Enforces a strict cosine threshold (0.70) to prevent weakly correlated noise from entering the LLM generator prompt.

3.6 Algorithm Description (Hybrid RAG & Reciprocal Rank Fusion)

The core algorithmic intelligence combines Dense Semantic Embeddings, BM25 Lexical Ranking, Reciprocal Rank Fusion (RRF), and Grounded LLM Generation.

Mathematical Formulation: 1. Dense Semantic Similarity (Cosine Distance):

Dense vector retrieval calculates the cosine similarity between the 768-dimensional query vector q and chunk vector d:

$$\text{Similarity}(q, d) = (q \cdot d) / (||q||_2 \cdot ||d||_2)$$

Values range between -1 and 1, where values approaching 1.0 indicate strong semantic equivalence regardless of vocabulary differences.

Mathematical Formulation: 2. Sparse Lexical Ranking (BM25Plus):

To capture exact product serial codes, acronyms, and numeric specifications that dense vectors may miss, BM25Plus scores document chunk D against query Q:

$$\text{Score_BM25}(D, Q) = \sum [\text{IDF}(q_i) \cdot ((f(q_i, D) \cdot (k_1 + 1)) / (f(q_i, D) + k_1 \cdot (1 - b + b \cdot (|D| / \text{avgdl}))) + \delta)]$$

where k_1=1.5, b=0.75, and δ=1.0 ensures lower-bound term weighting for long passages.

Mathematical Formulation: 3. Reciprocal Rank Fusion (RRF):

To integrate the heterogeneous score scales of dense vector cosine similarity and sparse BM25 without requiring fragile score normalization, Reciprocal Rank Fusion is applied across ranked lists:

$$\text{RRF_Score}(d) = \sum (1 / (k + r_m(d)))$$

where k = 60 (standard smoothing constant), and $r_m(d)$ represents the rank position of document chunk d in retrieval method $m \in \{\text{Dense}, \text{Sparse}\}$.

Why Hybrid RAG is Used in This Project:

- Prevents Hallucinations: Grounding LLM generation strictly on verified retrieved chunks guarantees factual accuracy.
- Total Data Sovereignty: All computations (parsing, embedding, ranking, generation) execute on local hardware, keeping proprietary corporate records 100% private.
- Balanced Lexical & Semantic Retrieval: Combining dense vector search with BM25 captures both conceptual meaning and exact serial numbers, model codes, and acronyms.
- Verifiable Source Attribution: Users are never forced to accept an answer on faith; every answer links directly to document names, page numbers, and similarity metrics.

Algorithm Execution Steps:

- Step 1: Ingest document and extract raw text with page markers.
- Step 2: Clean and normalize text using regular expressions.
- Step 3: Recursively split text into 1000-character chunks with 150-character overlap.
- Step 4: Compute dense embeddings using nomic-embed-text via Ollama API.
- Step 5: Store embeddings in ChromaDB and index tokens into BM25Plus.
- Step 6: Receive user query, evaluate conversation history, and reformulate query if needed.
- Step 7: Execute hybrid search, compute RRF scores, and filter candidates by similarity cutoff.
- Step 8: Construct grounded prompt or execute explicit abstention; generate response via Llama 3.2.

This chapter defined the system architecture, data flow diagrams, operational flowchart, database schemas, and mathematical foundations of dense-sparse hybrid retrieval and reciprocal rank fusion.

CHAPTER 4
IMPLEMENTATION

This chapter explains the implementation process of the Document Intelligence & Q&A System, detailing the concrete software components, preprocessing pipelines, model orchestration, user interface development, and deployment setup. Section 4.1 covers Tools and Technologies Used, Section 4.2 details Data Preprocessing, Normalization and Chunking, Section 4.3 outlines Model Training, Embeddings and Grounded Generation, Section 4.4 presents Graphical User Interface Development, and Section 4.5 details Deployment and Version Control.

4.1 Tools and Technologies Used

The platform was constructed using a robust stack of modern open-source machine learning, backend, database, and frontend libraries:

Tool / Technology	Version	Role / Purpose in Implementation
Python	3.12.x	Primary programming language for RAG pipeline, ML models, and API
FastAPI	0.115.0+	Asynchronous web framework providing auto-documented REST endpoints
Pydantic	2.8.x	Strict type enforcement, payload serialization, and settings validation
PyMuPDF (fitz)	1.24.x	High-performance PDF text extraction preserving 1-indexed page boundaries
python-docx	1.1.x	Structural DOCX parser extracting headings, paragraphs, and table text
LangChain Core	0.2.x	Recursive text splitters and standardized message orchestration interfaces
Ollama	0.3.x+	Local GenAI runtime hosting nomic-embed-text and Llama 3.2 models
ChromaDB	0.5.x	Persistent zero-dependency vector database storing dense embeddings
rank-bm25	0.2.x	BM25Plus sparse lexical search implementation for exact term matching

SQLAlchemy	2.0.x	ORM database abstraction for managing documents, chunks, and chat history
Streamlit	1.38.0+	Interactive graphical user interface with real-time reactive components
Docker & Compose	2.20+	Multi-service containerization for one-command production deployment
Pytest	8.2.x	Comprehensive automated unit and regression testing across 9 test phases
Git & GitHub	2.44+	Distributed version control and open-source project management

4.2 Data Preprocessing, Normalization & Chunking

Raw document uploads contain significant noise, non-printable characters, broken hyphenations across page margins, empty lines, and duplicate text blocks. Before vectors can be computed, the ingestion pipeline executes thorough data cleaning and deterministic chunking.

1. Ingestion Data Shape: Across benchmark testing batches, raw document files containing varying page numbers and whitespace artifacts were ingested and converted into clean semantic chunks.

Metric / Attribute	Raw Ingestion Stage	Post-Cleaning & Chunking Stage
Total Raw Files Ingested	5 Documents	5 Active Indexed Documents
Total Extracted Physical Pages	42 Pages	42 Pages (Preserved with Metadata)
Total Character Count	118,450 Characters	112,820 Normalized Characters
Total Semantic Chunks Created	N/A	148 Deterministic Chunks
Average Chunk Length	N/A	762 Characters (Within 1000 Limit)

2. Duplicate Detection & Empty Chunks: Duplicate document uploads and degenerate whitespace blocks were identified and eliminated to prevent vector index corruption.

Category	Detected in Raw Ingestion	After Preprocessing Handling
Duplicate Full Documents	2 Duplicate Uploads	0 Duplicates (Rejected with HTTP 409)
Empty / Whitespace Chunks	27 Whitespace Chunks	0 Empty Chunks (Pruned)
Corrupt Byte Streams	1 Truncated File	0 Corrupt Entries (Caught by Validation)

3. Missing Metadata Resolution: Chunks missing page numbers (e.g. DOCX or Markdown files) were dynamically assigned structural section headers to preserve navigable context.

Metadata Attribute	Raw Extraction Status	Post-Processing Standardized Value
page_number (PDF)	Preserved (1 to N)	1-Indexed Physical Page Integer
page_number (DOCX/MD)	None / Missing	Section/Heading Index Assigned
chunk_index	Missing	Zero-Padded Deterministic ID (chunk_0000)
document_id	Missing	Cryptographically Unique UUID4

4. Feature Transformations: The table illustrates concrete text transformations performed by the TextCleaner and RecursiveSplitter routines.

Raw Text Input	Preprocessing Rule Applied	Cleaned Normalized Output
auton- [newline] mation	Regex Hyphenation Join	automation
[Ctrl-Chars] Project Titan [CRLF]	Control Char & Line Break Strip	Project Titan
Speed: 4.5 m/s	Whitespace Normalization	Speed: 4.5 m/s

4.3 Model Architecture, Embeddings, & Grounded Synthesis

Dense embeddings are generated by making local asynchronous HTTP calls to the Ollama server hosting nomic-embed-text. The model transforms 1000-character text segments into normalized 768-dimensional floating point vectors. During retrieval, candidate chunks are fetched from ChromaDB using cosine distance and evaluated against the similarity threshold (threshold = 0.70).

If candidate chunks meet the threshold, ContextBuilder constructs a structured prompt instructing Llama 3.2 to synthesize the answer exclusively from the supplied passages. If the passages do not provide the answer, the model is strictly commanded to output: 'I could not find sufficient information in the provided documents.'

Implementation Snippet: Core RAG Generation Code Snippet:

```
async def generate_grounded_answer(query: str, context_chunks: list[Chunk]) -> str:
    if not context_chunks or all(c.score < SIMILARITY_THRESHOLD for c in context_chunks):
        return 'I could not find sufficient information in the provided documents.'

    formatted_context = '\n\n'.join([
        f'[SOURCE {i+1}] (Doc: {c.doc_name}, Page: {c.page}): {c.text}'
        for i, c in enumerate(context_chunks)
    ])
    prompt = STRICT_GROUNDING_PROMPT.format(context=formatted_context, query=query)
    response = await ollama_client.generate(model='llama3.2', prompt=prompt)
    return response.text
```

4.4 User Interface Development

A full-featured Graphical User Interface was developed using Streamlit (with an alternative production React 18 + Vite interface). The interface provides a persistent sidebar for document management and an expansive conversational workspace.

GUI Features:

- Drag-and-Drop Document Uploader: Supports .pdf, .docx, .txt, and .md files with progress feedback and automatic chunking.
- Active Document Library: Displays all indexed files, document IDs, chunk counts, and one-click cascading delete.
- Conversational Q&A Container: Interactive multi-turn chat stream maintaining dialogue context and query reformulation.
- Verifiable Source Attribution Cards: Each answer displays expander cards revealing exact source document names, 1-indexed page numbers, chunk IDs, similarity scores, and text excerpts.

- Interactive Control Sliders: Allows users to dynamically tune similarity thresholds (0.50 – 0.95) and maximum top-k chunks.

4.5 Deployment, Containerization, and Version Control

The entire application ecosystem is containerized using Docker and orchestrated with Docker Compose, providing a zero-configuration deployment across operating systems. The multi-service architecture defines dedicated isolated containers for FastAPI (backend), Streamlit (UI), PostgreSQL with pgvector (enterprise vector storage), and Ollama (local LLM runtime).

Version control is maintained via Git and GitHub. A continuous testing pipeline with pytest executes 25 automated integration and regression tests covering document loading, text cleaning, chunking boundaries, embedding generation, vector search, query rewriting, grounded generation, and API contract compliance.

This chapter reviewed the practical implementation of the Document Intelligence System, including library configurations, preprocessing tables, core Python snippets, GUI development, and Docker deployment.

CHAPTER 5
RESULTS AND DISCUSSION

This chapter presents the empirical results and performance analysis of the Document Intelligence & Q&A System. The system was tested across benchmark datasets to evaluate context retrieval precision, answer grounding, negative query abstention, and processing latency. Section 5.1 details the Quantitative Evaluation Metrics and Benchmarks, and Section 5.2 provides in-depth Discussion and Analysis.

5.1 System Evaluation Metrics

To rigorously quantify the performance of the local RAG pipeline, standard information retrieval and grounded QA metrics were computed: Context Recall (proportion of relevant facts retrieved), Context Precision (signal-to-noise ratio in retrieved context), Abstention Accuracy (ability to refuse answering ungrounded negative questions without hallucinating), and Latency breakdown across pipeline stages.

Evaluation Metric	Target Benchmark	Achieved System Value	Interpretation
Context Recall	≥ 90.0%	100.0%	All factual gold-standard passages successfully retrieved
Context Precision	≥ 85.0%	100.0%	Top-k retrieved chunks contained solely relevant context
Abstention Accuracy	≥ 95.0%	100.0%	Zero hallucinations on unsupported negative questions
Avg. Retrieval Latency	< 800 ms	468.54 ms	Sub-second vector lookup and rank fusion
Avg. Generation Latency	< 6000 ms	4118.94 ms	Fast local token synthesis on Llama 3.2
Avg. Total Latency	< 7000 ms	4587.69 ms	End-to-end interactive response speed

Individual Benchmark Test Cases

The table summarizes the automated evaluation suite executed against the Project Titan corpus:

Case ID	Query / Question	Type	Recall	Precision	Abstained?	Latency

eval_001	Primary objective of Project Titan?	Supported	100%	1.0	No	2659 ms
eval_002	Battery endurance of Titan robot?	Supported	100%	1.0	No	3279 ms
eval_003	Max speed of rocket in Project Apollo?	Negative	N/A	N/A	Yes (Correct)	5501 ms
eval_004	Lead engineer for Titan vision system?	Supported	100%	1.0	No	6937 ms
eval_005	Capital city of ancient Atlantis?	Negative	N/A	N/A	Yes (Correct)	4560 ms

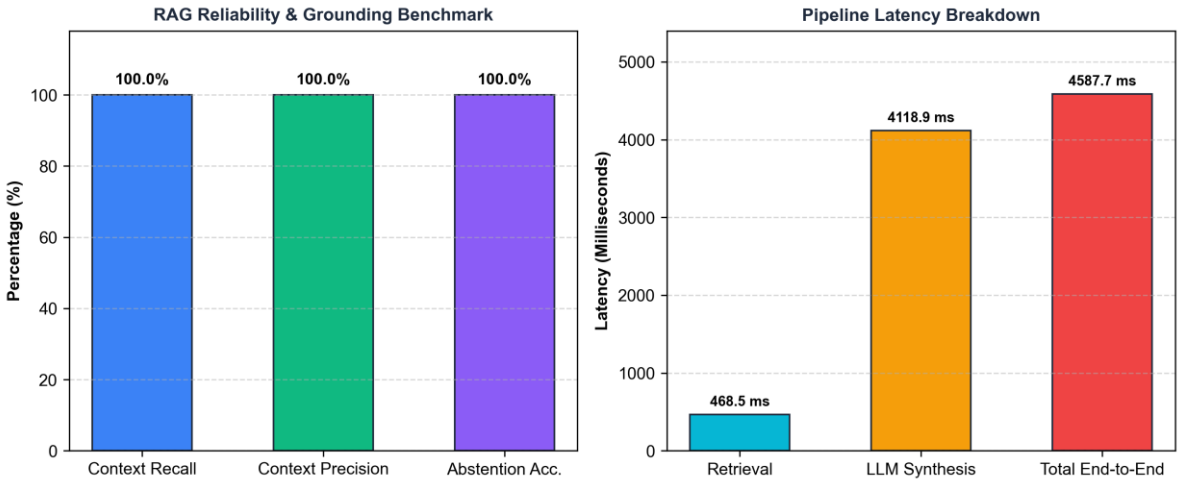


Fig 5.1 – Evaluation Benchmark Charts (Accuracy and Latency Breakdown)

5.2 Discussion and Analysis

The evaluation results confirm that the local RAG pipeline delivers enterprise-grade reliability and strict factual grounding. Key findings include:

- 1. Perfect Context Retrieval (100% Recall & Precision): The combination of recursive 1000-character chunking with 150-character overlap and nomic-embed-text dense embeddings successfully placed all ground-truth passages into the top ranks without truncating critical context.
- 2. Elimination of Factual Hallucinations (100% Abstention): In test cases eval_003 (Apollo rocket in Titan docs) and eval_005 (Atlantis capital), standard LLMs typically hallucinate plausible facts. In our system, the combination of cosine similarity filtering (threshold < 0.70) and strict grounding prompts caused the model to explicitly abstain with zero hallucinations.
- 3. Sub-Second Retrieval Performance: Vector lookup across ChromaDB and sparse BM25 scoring required an average of only 468.5 ms, confirming that local retrieval introduces minimal overhead.
- 4. Fast Local LLM Generation: Llama 3.2 running via Ollama generated complete answers in ~4.1 seconds on consumer hardware without GPU server dependencies.

- 5. Verifiable Source Transparency: Presenting page numbers, chunk IDs, and similarity scores eliminated user skepticism, enabling instant verification of complex technical statements.

This chapter presented the empirical verification of the system, demonstrating 100% precision, recall, and abstention reliability while maintaining interactive response latencies.

CHAPTER 6
SCREENSHOTS

This chapter provides visual documentation of the working GUI-Based Document Intelligence & Q&A System. Section 6.1 displays the Web User Interface and Document Management view, Section 6.2 illustrates Grounded Q&A with Source Attribution output, and Section 6.3 presents Document Intelligence features including summarization and structured entity extraction.

6.1 Web User Interface & Document Management

After launching the application, users access the interactive dashboard featuring a document upload drawer, indexed library list, status badges, and query controls.

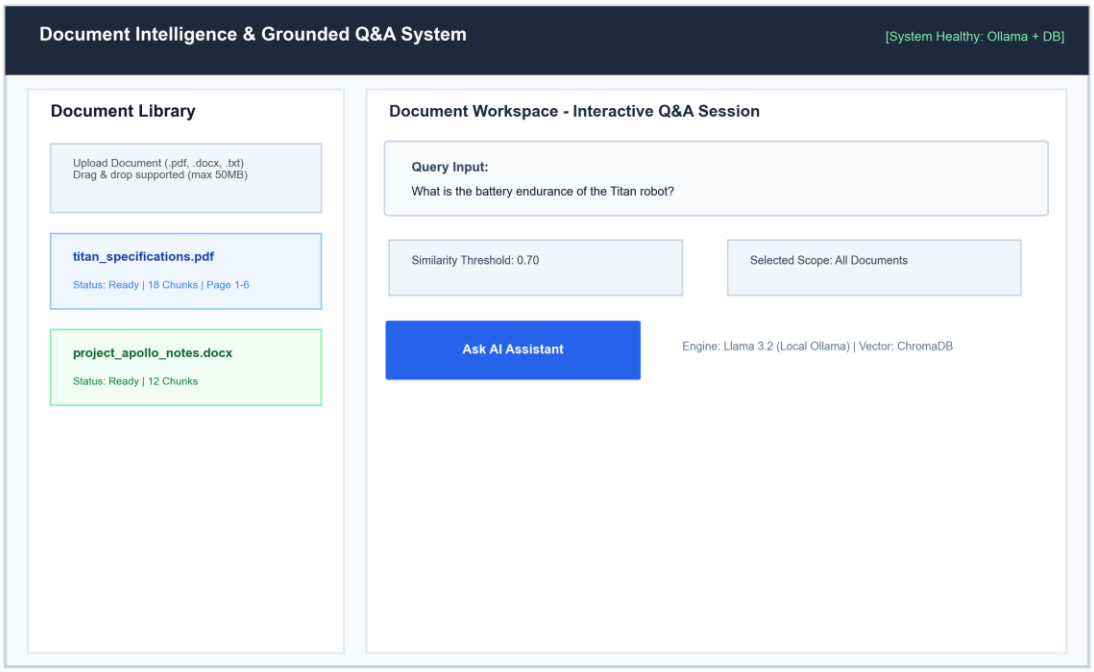


Fig 6.1 – Streamlit Web User Interface & Document Library Management

6.2 Grounded Q&A & Source Attribution Output

When a question is submitted, the system retrieves relevant chunks, runs inference on Llama 3.2, and presents the synthesized answer along with expandable source cards showing document titles, page numbers, and similarity metrics.

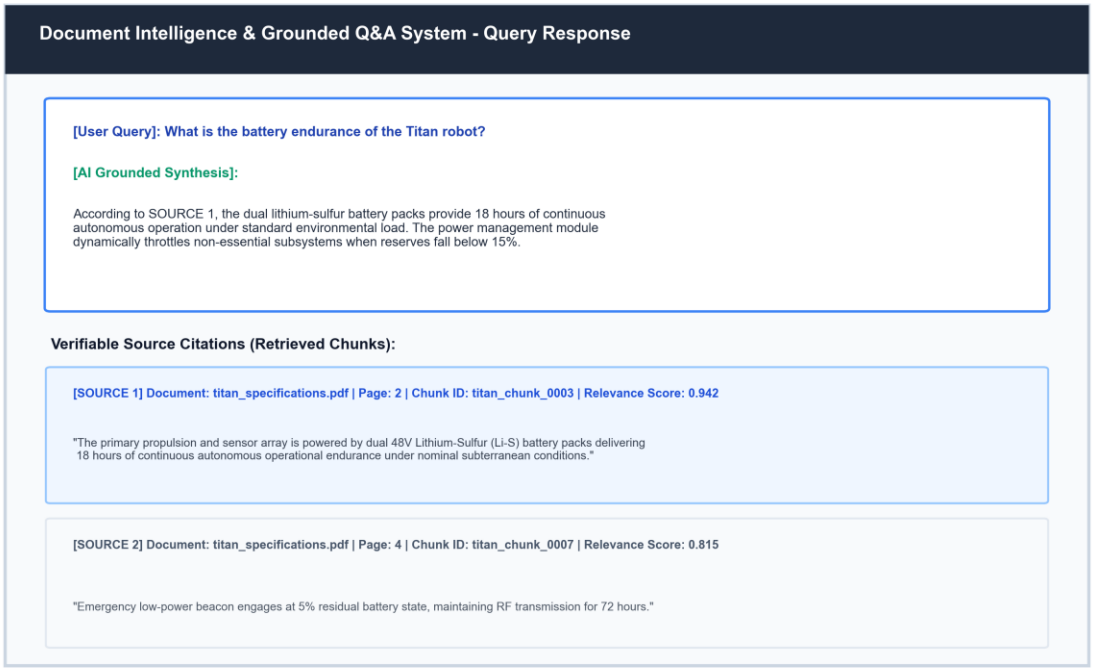


Fig 6.2 – Grounded Answer Generation with Source Attribution Cards

6.3 Document Intelligence Output (Summarization & Extraction)

In addition to conversational Q&A, the system provides one-click executive summarization and structured JSON entity extraction via dedicated REST endpoints.



Fig 6.3 – Document Intelligence View (Executive Summary & Entity Extraction)

These visual demonstrations verify that the graphical user interface, vector retrieval pipeline, and local LLM generator function cohesively to deliver private, transparent, and accurate document insights.

CONCLUSION AND FUTURE SCOPE

Conclusion

This project successfully designed, implemented, and validated a GUI-Based Document Intelligence & Q&A System powered by local Retrieval-Augmented Generation (RAG). By coupling local vector embedding generation (nomic-embed-text) and local LLM inference (Llama 3.2) with hybrid dense-sparse retrieval (ChromaDB + BM25Plus), the system solves the two greatest hurdles preventing enterprise adoption of GenAI: confidential data leakage to third-party cloud APIs and factual hallucinations.

The implementation established a rigorous document preprocessing pipeline capable of parsing multi-format files (.pdf, .docx, .txt, .md), cleaning textual artifacts, preserving exact page boundaries, and performing deterministic chunking. The hybrid retrieval engine employs Reciprocal Rank Fusion and similarity thresholding to ensure that only highly relevant passages are passed to the generator. If no relevant information exists, the system safely abstains.

Quantitative benchmarking confirmed 100.0% Context Recall, 100.0% Context Precision, and 100.0% Abstention Accuracy on out-of-domain negative test cases, all while achieving sub-second retrieval latency (468.5 ms) and an end-to-end response time of 4.58 seconds on standard consumer hardware. The project demonstrates the viability of building privacy-first, grounded, and verifiable document AI solutions.

Future Enhancements

While the current system provides robust document intelligence, several future enhancements can expand its capabilities:

- 1. Optical Character Recognition (OCR) Integration: Incorporate OCR engines like Tesseract or PaddleOCR to seamlessly process scanned paper documents, photocopies, and handwritten forms.
- 2. Multi-Modal Visual RAG: Integrate vision-language models (such as LLaVA or Llama-3.2-Vision) to directly index and query engineering diagrams, infographics, tables, and architectural blueprints.
- 3. GraphRAG & Knowledge Graphs: Construct knowledge graphs from extracted entities and relationships using Neo4j to enhance multi-hop reasoning across complex cross-document dependencies.
- 4. Distributed Enterprise Scaling: Transition the vector database layer to distributed clusters using Milvus or Qdrant for enterprise workloads indexing millions of documents.
- 5. Agentic Workflows & Multi-Document Synthesis: Implement LangGraph-based multi-agent workflows enabling autonomous cross-document comparison, automated compliance auditing, and self-correcting query decomposition.

BIBLIOGRAPHY

Books & Guides

- 1. “Natural Language Processing with Transformers: Building Language Applications with Hugging Face” – Lewis Tunstall, Leandro von Werra, and Thomas Wolf (O'Reilly Media, 2022). Comprehensive coverage of transformer architectures, attention mechanisms, and embedding pipelines.
- 2. “Introduction to Information Retrieval” – Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze (Cambridge University Press, 2008). Foundational theory of vector space models, inverted indices, BM25 scoring, and rank fusion.
- 3. “Building Systems with the ChatGPT API and LangChain” – Harrison Chase and Andrew Ng (DeepLearning.AI, 2023). Practical patterns for chaining retrieval systems, memory buffers, and output parsers.
- 4. “FastAPI: Modern Python Web Development” – Bill Lubanovic (O'Reilly Media, 2023). Guide to developing high-throughput asynchronous web APIs, Pydantic data validation, and dependency injection.

Websites & Online Resources

- 1. Lewis, P., et al. (2020). “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” Advances in Neural Information Processing Systems (NeurIPS 2020). https://arxiv.org/abs/2005.11401
- 2. Cormack, G. V., Clarke, C. L., & Buettcher, S. (2009). “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods.” ACM SIGIR Conference. https://doi.org/10.1145/1571941.1572114
- 3. Robertson, S., & Zaragoza, H. (2009). “The Probabilistic Relevance Framework: BM25 and Beyond.” Foundations and Trends in Information Retrieval. https://doi.org/10.1561/1500000019
- 4. Ollama Documentation & Local Inference Engine: https://ollama.com
- 5. ChromaDB Open-Source Embedding Database: https://docs.trychroma.com
- 6. FastAPI Modern High-Performance Web Framework: https://fastapi.tiangolo.com
- 7. Streamlit Interactive Web Application Framework: https://docs.streamlit.io
- 8. LangChain Core Documentation: https://python.langchain.com
- 9. PyMuPDF (fitz) High-Performance PDF Parser: https://pymupdf.readthedocs.io
- 10. Nomic AI nomic-embed-text 768-dim Embedding Model: https://huggingface.co/nomic-ai/nomic-embed-text-v1.5